

7 Basic principles of testing**Testing shows the presence of defects**

This principle of testing states that testing aims to highlight the presence of defects so that developers can rectify them. Software testing can ensure that defects are present but it cannot prove that software is defect-free. Even multiple testing can never ensure that software is 100% bug-free. Testing can reduce the number of defects but not remove all defects.

Exhaustive testing is impossible

It is the process of testing the functionality of the software in all possible inputs (valid or invalid) and pre-conditions is known as exhaustive testing. Exhaustive testing is impossible means the software can never test at every test case. It can test only some test cases and assume that the software is correct and it will produce the correct output in every test case. If the software will test every test case then it will take more cost, effort, etc., which is impractical.

Early Testing: To find the defect in the software, early test activity shall be started. The defect detected in the early phases of SDLC will be very less expensive. For better performance of software, software testing will start at the initial phase i.e. testing will perform at the requirement analysis phase.

Defect clustering: In a project, a small number of modules can contain most of the defects. Pareto Principle to software testing state that 80% of software defect comes from 20% of modules.

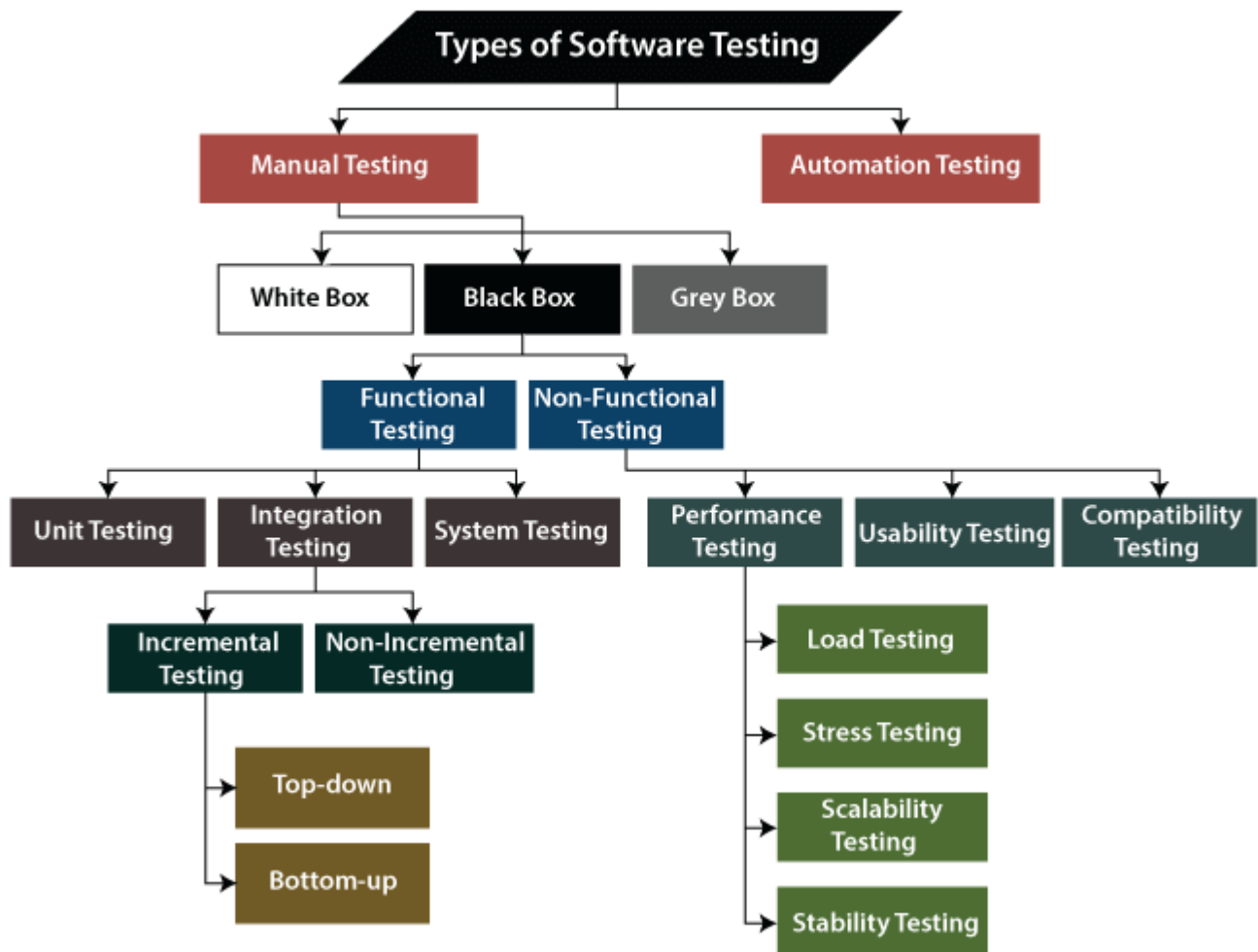
Pesticide paradox: Repeating the same test cases, again and again, will not find new bugs. So it is necessary to review the test cases and add or update test cases to find new bugs.

Testing is context-dependent: The testing approach depends on the context of the software developed. Different types of software need to perform different types of testing. For example, The testing of the e-commerce site is different from the testing of the Android application.

Absence of errors fallacy: If a built software is 99% bug-free but it does not follow the user requirement then it is unusable. It is not only necessary that software is 99% bug-free but it is also mandatory to fulfill all the customer requirements.

Types of Software Testing

If we want to ensure that our software is bug-free or stable, we must perform the various types of software testing because testing is the only method that makes our application bug-free.



The different types of Software Testing

The software testing mainly divided into two parts, which are as follows:

- 1) **Manual Testing**
- 2) **Automation Testing**

1) Manual Testing

Testing any software or an application according to the client's needs without using any automation tool is known as **manual testing**.

We do not require any precise knowledge of any testing tool to execute the manual test cases.

Classification of Manual Testing

In software testing, manual testing can be further classified into **three different types of testing**, which are as follows:

- a) **White Box Testing**
- b) **Black Box Testing**
- c) **Grey Box Testing**

a) White Box Testing

In white-box testing, the developer will inspect every line of code before handing it over to the testing team or the concerned test engineers.

In other words, we can say that the **developer** will execute the complete white-box testing for the particular software and send the specific application to the testing team.

White box testing is also known as **open box testing, glass box testing, structural testing, clear box testing, and transparent box testing.**

b) Black Box Testing

Another type of manual testing is **black-box testing**. In this testing, the test engineer will analyze the software against requirements, identify the defects or bug, and sends it back to the development team.

Then, the developers will fix those defects, do one round of White box testing, and send it to the testing team.

Here, fixing the bugs means the defect is resolved, and the particular feature is working according to the given requirement.

In other words, we can say that black box testing is a process of checking the functionality of an application as per the customer requirement. The source code is not visible in this testing; that's why it is known as **black-box testing**.

Types of Black Box Testing

Black box testing further categorizes into two parts, which are as discussed below:

- i. **Functional Testing**
- ii. **Non-function Testing**

i) Functional Testing

The test engineer will check all the components systematically against requirement specifications is known as **functional testing**. Functional testing is also known as **Component testing**.

In functional testing, all the components are tested by giving the value, defining the output, and validating the actual output with the expected value.

Types of Functional Testing

The diverse **types of Functional Testing** contain the following:

- **Unit Testing**
- **Integration Testing**
- **System Testing**

1. Unit Testing

Unit testing is the first level of functional testing in order to test any software. In this, the test engineer will test the module of an application independently or test all the module functionality is called **unit testing**.

2. Integration Testing

Once we are successfully implementing the unit testing, we will do integration testing. It is the second level of functional testing, where we test the data flow between dependent modules or interface between two features is called **integration testing**.

Types of Integration Testing

Integration testing is also further divided into the following parts:

- **Incremental Integration Testing**
- **Non-Incremental Integration Testing**

Incremental Integration Testing

Whenever there is a clear relationship between modules, we go for incremental integration testing. Suppose, we take two modules and analysis the data flow between them if they are working fine or not.

If these modules are working fine, then we can add one more module and test again. And we can continue with the same process to get better results.

In other words, we can say that incrementally adding up the modules and test the data flow between the modules is known as **Incremental integration testing**.

Types of Incremental Integration Testing

Incremental integration testing can further classify into two parts, which are as follows:

1. **Top-down Incremental Integration Testing**
2. **Bottom-up Incremental Integration Testing**

1. Top-down Incremental Integration Testing

In this approach, we will add the modules step by step or incrementally and test the data flow between them. We have to ensure that the modules we are adding are the **child of the earlier ones**.

2. Bottom-up Incremental Integration Testing

In the bottom-up approach, we will add the modules incrementally and check the data flow between modules. And also, ensure that the module we are adding is the **parent of the earlier ones**.

Non-Incremental Integration Testing/ Big Bang Method

Whenever the data flow is complex and very difficult to classify a parent and a child, we will go for the non-incremental integration approach. The non-incremental method is also known as **the Big Bang method**.

3. System Testing

Whenever we are done with the unit and integration testing, we can proceed with the system testing.

In system testing, the test environment is parallel to the production environment. It is also known as **end-to-end** testing.

In this type of testing, we will undergo each attribute of the software and test if the end feature works according to the business requirement and analyze the software product as a complete system.

ii) Non-functional Testing

The next part of black-box testing is **non-functional testing**. It provides detailed information on software product performance and used technologies.

Non-functional testing is a combination of **performance, load, stress, usability and, compatibility testing**.

Types of Non-functional Testing

1. **Performance Testing**
2. **Usability Testing**
3. **Compatibility Testing**

1. Performance Testing

In performance testing, the test engineer will test the working of an application by applying some load.

Classification of Performance Testing

- **Load Testing**
- **Stress Testing**

- **Scalability Testing**
- **Stability Testing**

Load Testing

While executing the performance testing, we will apply some load on the particular application to check the application's performance, known as **load testing**. Here, the load could be less than or equal to the desired load.

Stress Testing

It is used to analyze the user-friendliness and robustness of the software beyond the common functional limits.

Scalability Testing

To analysis, the application's performance by enhancing or reducing the load in particular balances is known as **scalability testing**.

Stability Testing

Stability testing is a procedure where we evaluate the application's performance by applying the load for a precise time.

2. Usability Testing

Another type of **non-functional testing** is **usability testing**. In usability testing, we will analyze the user-friendliness of an application and detect the bugs in the software's end-user interface.

Here, the term **user-friendliness** defines the following aspects of an application:

- The application should be easy to understand, which means that all the features must be visible to end-users.
- The application's look and feel should be good that means the application should be pleasant looking and make a feel to the end-user to use it.

3. Compatibility Testing

In compatibility testing, we will check the functionality of an application in specific hardware and software environments. Once the application is functionally stable then only, we go for **compatibility testing**.

Here, we can test the application on the different operating systems and other browsers, and **hardware** means we can test the application on different sizes.

iii) Grey Box Testing

Another part of **manual testing** is **Grey box testing**. It is a **collaboration of black box and white box testing**.

Since, the grey box testing includes access to internal coding for designing test cases. Grey box testing is performed by a person who knows coding as well as testing.



In other words, we can say that if a single-person team done both **white box and black-box testing**, it is considered **grey box testing**.

2) Automation Testing

The most significant part of Software testing is Automation testing. It uses specific tools to automate manual design test cases without any human interference.

Automation testing is the best way to enhance the efficiency, productivity, and coverage of Software testing.

In other words, we can say that whenever we are testing an application by using some tools is known as **automation testing**.

Need of testing

- Testing should be introduced in the early stage of the SDLC, The cost of fixing the bug is larger if testing is not done in early-stage & bugs found in later stages.
- In today's competitive market only the quality product stays longtime firmly, so to make sure they produce a good quality product the testing of application is a key factor in SDLC.
- The testing is important since it discovers defects/bugs before the delivery to the client, which guarantees the quality of the software.
- It makes the software more reliable and easy to use.
- Thoroughly tested software ensures reliable and high-performance software operation.

Testing process and activities

Testing is a process rather than a single activity. Testing must be planned and it requires discipline to act upon it. The quality and effectiveness of software testing are primarily determined by the quality of the test processes used. The activities of testing can be divided into the following basic steps:

1. Planning and Control
2. Analysis and Design
3. Implementation and Execution
4. Evaluating exit criteria and Reporting
5. Test Closure activities

1) Planning and Control

Test Planning: Test planning involves producing a document that describes an overall approach and test objectives. It involves reviewing the test basis, identifying the test conditions based on analysis of test items, writing test cases and Designing the test environment. Completion or exit criteria must be specified so that we know when testing (at any stage) is complete.

Control This is the activity of comparing actual progress against the plan, and reporting the status, including deviations from the plan. It involves taking actions necessary to meet the mission and objectives of the project.

2) Analysis and Design

Test analysis and Test Design has the following major tasks:

- To review the test basis. The test basis is the information on which test cases are based, such as requirements, design specifications, product risk analysis, architecture and interfaces
- To identify test conditions
- To design the tests
- To design the test environment set-up and identify the required infrastructure and tools

3) Implementation and Execution

Test execution involves actually running the specified test on a computer system either manually or by using an automated test tool. Test implementation has the following major task:

- To develop and prioritize test cases by using techniques and create test data for those tests.
- To create test suites from the test cases for efficient test execution. Test suite is a collection of test cases that are used to test a software program
- To re-execute the tests that previously failed in order to confirm a fix.
- To log the outcome of the test execution. A test log is the status of the test case (pass/fail).
- To compare actual results with expected results.

4) Evaluating Exit criteria and Reporting

Evaluating exit criteria is a process defining when to stop testing. It depends on coverage of code, functionality or risk. Basically it also depends on business risk, cost and time and vary from project to project.

5) Test Closure activities:

Test closure activities are done when software is ready to be delivered. The testing can be closed for the other reasons also like:

- When a project is cancelled
- When some target is achieved

Strategy of testing

A strategy of software testing is shown in the context of spiral.

Following figure shows the testing strategy:

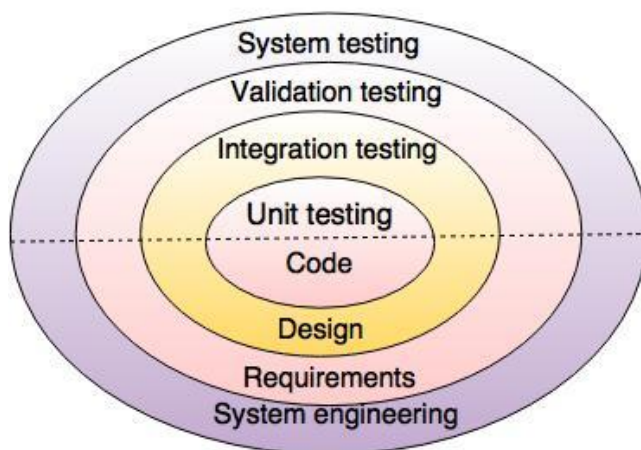


Fig. - Testing Strategy

Unit testing

Unit testing starts at the centre and each unit is implemented in source code.

Integration testing

An integration testing focuses on the construction and design of the software.

Validation testing

Check all the requirements like functional, behavioral and performance requirement are validate against the construction software.

System testing

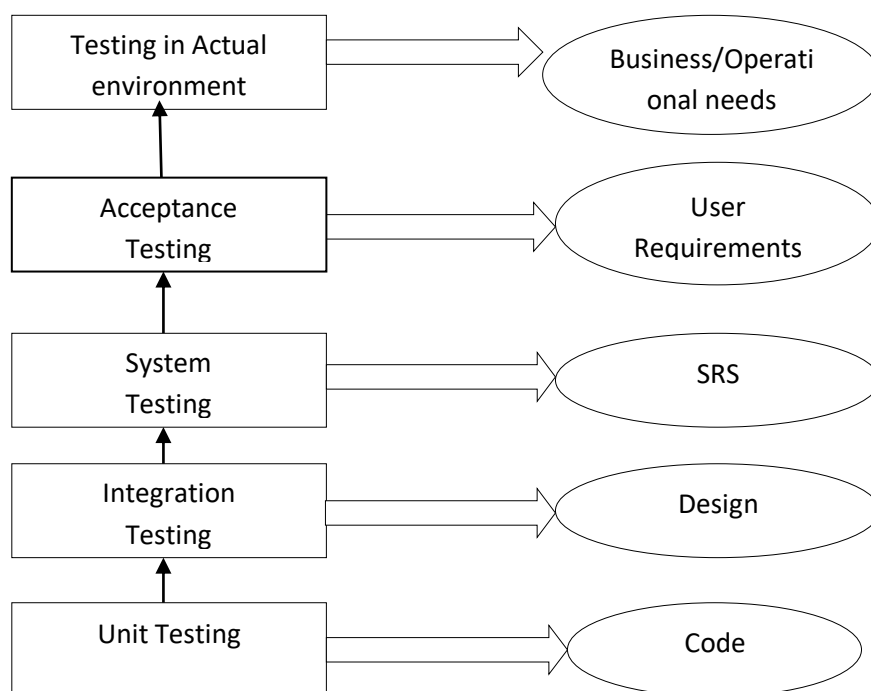
System testing confirms all system elements and performance are tested entirely.

An efficient testing strategy includes two types of tests, namely, low level tests and high level tests. Low level tests ensure correct implementation of small part of the source code and high level tests ensure that major software functions are validated according to user requirements.

A testing strategy sets certain milestones for the software such as final date for completion of testing and the date of delivering the software. These milestones are important when there is limited time to meet the deadline.

Testing Levels:

The following diagram shows the different levels of testing



Units/Components Testing:

Unit/component is the smallest entity in the software. Every unit has to be tested separately to check whether it is as per the specifications. But it is not possible to test a unit individually. Additional code may be written to test the units they are called stubs, drivers & simulators. Each function can be considered as a unit.

Unit/Component testing is done only by the development engineers. This testing is done to check the functionality as well as to ensure that the code is well written.

Module Testing:

A **module** is an independent entity in the software. The tested units are integrated into a module and each module is tested separately for the applications.

For example, if the s/w is a DBMS, the modules can be a Database (back-end) & the user interface (front-end). The database can be tested separately using the SQL commands to check correct data retrieval. But validation of front-end requires a database. So after testing the back-end module the front-end can be tested, after integration.

Integration Testing:

After the modules are tested, the modules can be integrated together and then testing is started. During integration testing, a very systematic way has to be followed. Foremost we need to identify which modules can be integrated first. For very large projects containing large number of modules, integration testing is the most challenging task. Once all the modules are integrated together, the system is ready for testing.

System Testing:

Once all the modules are integrated, the system is ready for testing. To start with the system has to be tested for its functionality. This is done by going through the SRS document & for each requirement carry out the test cases design, executes the test cases and compares the expected results with the actual results. This functional testing is different from application to application.

In addition to functional testing, the system has to be tested for its non-functional requirements such as:

- Performance requirements
- Accessibility to physically handicapped persons
- Portability
- Security
- Reliability
- Safety
- Interoperability with other h/w or s/w etc.

Acceptance Testing:

After the testing is completed (system testing) in the organization, the s/w has to be approved by the customer. The customer will carry out a rigorous (fast) testing called Acceptance Testing. If there are no defects found, then the customer will pay the money. If he found problems during this testing, the customer will be unhappy